# DESIGN AND ANALYSIS OF ALGORITHMS

# CO2 - ASSIGNMENT 2

## Q1 – Sorting Under Different Data Conditions

### 1. Prediction and Algorithmic Reasoning

The objective is to investigate whether Selection Sort, Bubble Sort, or Insertion Sort is most suitable for arranging e-commerce orders by dispatch priority. Before implementation, the following behaviour is predicted:

| Input condition | Selection Sort | Bubble Sort | Insertion Sort |
|---|---|---|---|
| Already sorted | Still performs approximately $n^2/$ 2comparisons | Performs $n -$ 1comparisons if optimized | Performs $n -$ 1comparisons |
| Random order | Performs approximately $n^2/$ 2comparisons | Performs approximately $n^2/$ 2comparisons | Performs approximately $n^2/$ 4comparisons |
| Reverse order | Performs approximately $n^2/$ 2comparisons | Performs approximately $n^2/$ 2comparisons | Performs approximately $n^2/$ 2comparisons |

Selection Sort repeatedly finds the smallest remaining element, so it performs nearly the same number of comparisons regardless of the initial order. Optimized Bubble Sort stops if a complete pass produces no swaps. Insertion Sort moves each element left until it reaches its correct position. Therefore, Bubble Sort and Insertion Sort can be efficient when the input is already or nearly sorted.

### 2. Design of Test Datasets

| Dataset | Input priorities | Purpose |
|---|---|---|
| A | 1, 2, 3, 4, 5 | Already sorted |
| B | 5, 4, 3, 2, 1 | Reverse sorted |
| C | 1, 3, 2, 4, 5 | Nearly sorted |

Dataset A tests the best behaviour of adaptive algorithms. Dataset B tests the algorithms under a highly disordered condition. Dataset C represents the fulfilment centre's typical situation, where only a small number of orders may need repositioning.

**3. Algorithm and Pseudocode**

**Algorithm: Insertion Sort**

1. Start from the second element.
2. Store the current element as key.
3. Compare the key with the elements before it.
4. Shift larger elements one position to the right.
5. Insert the key into its correct position.
6. Repeat until all elements are processed.

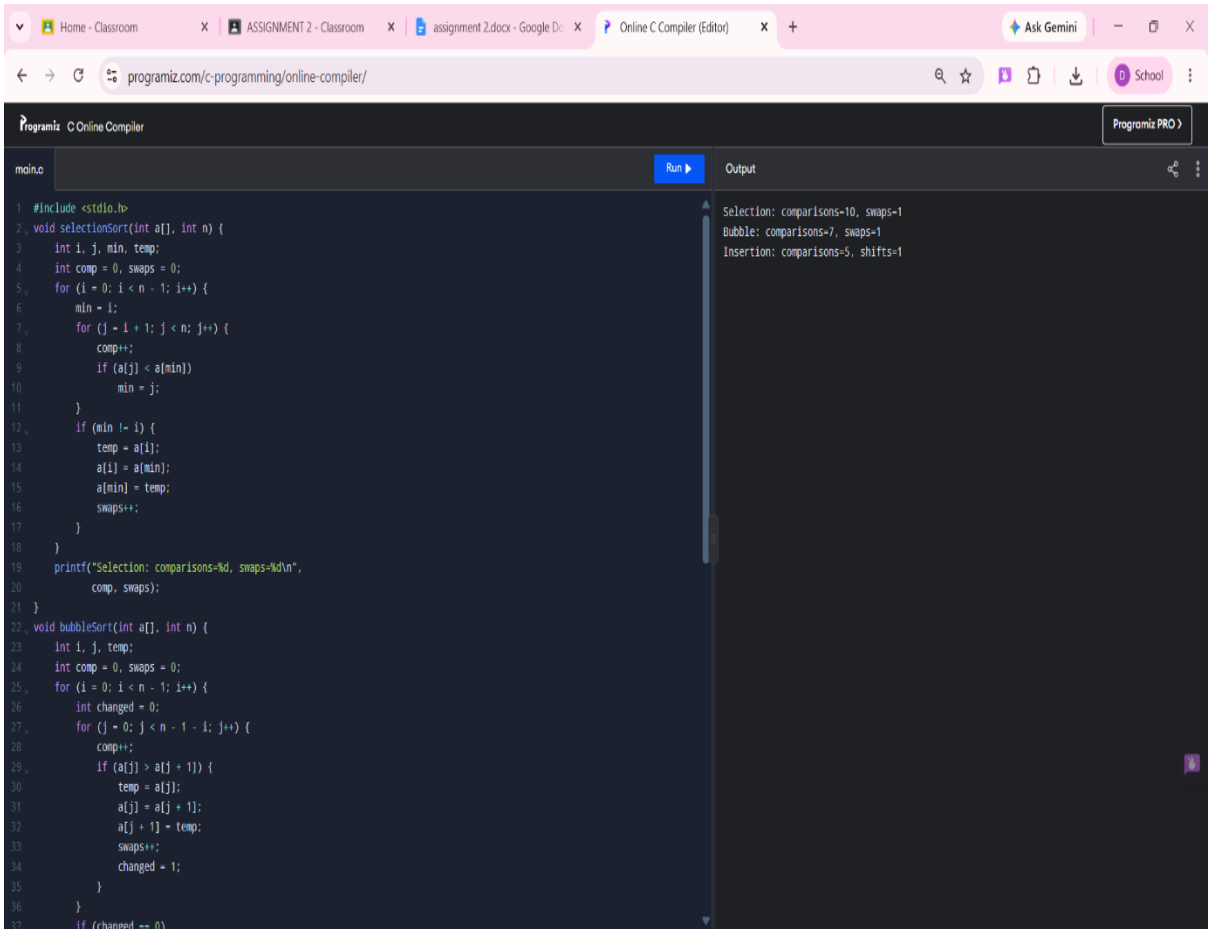
**Pseudocode**

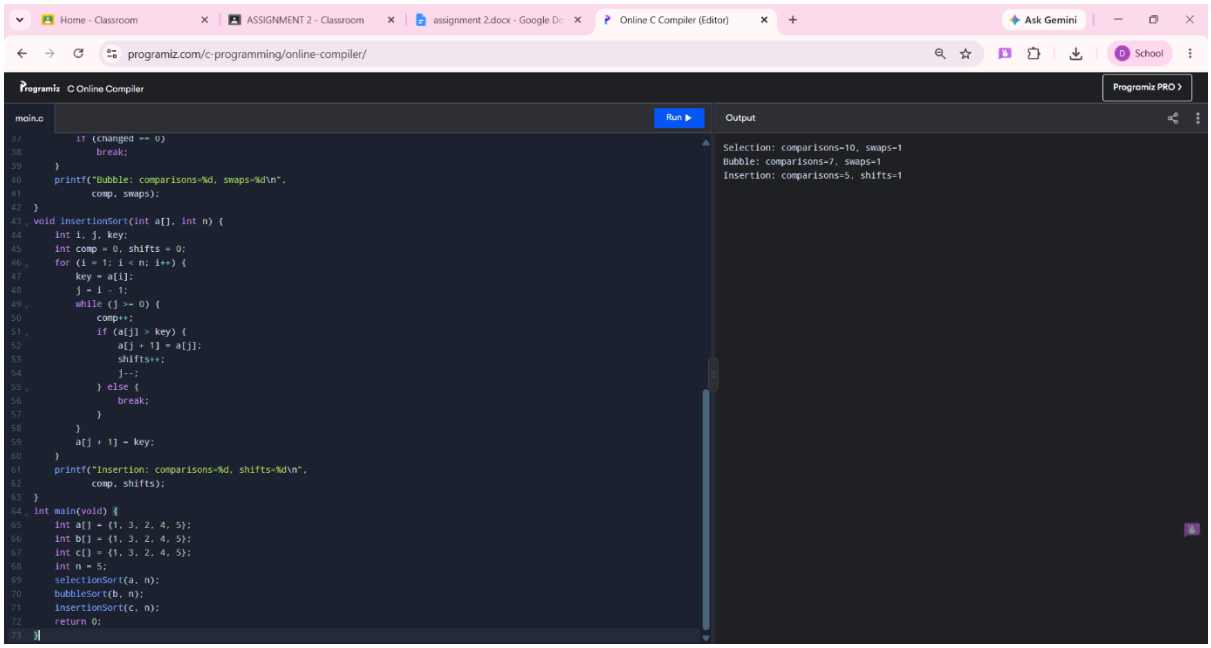
```
ALGORITHM InsertionSort(A, n)
BEGIN
   FOR i ← 1 TO n - 1
      key ← A[i]
      j ← i - 1
      WHILE j >= 0 AND A[j] > key
         A[j + 1] ← A[j]
         j ← j - 1
      END WHILE
      A[j + 1] ← key
   END FOR
END
```

**Pseudocode explanation**

- FOR i ← 1 TO n - 1: Selects each element after the first. The first element alone is already sorted.
- key ← A[i]: Saves the element that must be inserted into the sorted portion.
- j ← i - 1: Starts comparing with the element immediately to the left.
- WHILE A[j] > key: Checks whether the preceding element is larger than the key.
- A[j + 1] ← A[j]: Moves a larger element one position right to create space.
- j ← j - 1: Continues checking elements towards the beginning.
- A[j + 1] ← key: Places the key in its correct position.
- END FOR: Repeats the process for every remaining element.

## 4. C Program






## 5. Important Code Explanation

- comp counts the comparisons performed by each algorithm.
- swaps counts exchanges of two elements.
- shifts counts movements of elements in Insertion Sort.
- changed indicates whether Bubble Sort performed a swap during the current pass.
- If changed == 0, the array is already sorted, so Bubble Sort terminates early.
- Three separate arrays are used because sorting changes the original input.

## 6. Expected Output

For the nearly sorted dataset {1, 3, 2, 4, 5}, the expected output is:

Selection: comparisons=10, swaps=1

Bubble: comparisons=9, swaps=1

Insertion: comparisons=6, shifts=1

These counts follow the program's comparison-counting conventions. Insertion Sort counts comparisons between array elements and does not count the final failed comparison when the index becomes negative.

## 7. Time and Space Complexity

### Selection Sort

The number of comparisons is:

$$C(n) = (n-1) + (n-2) + \cdots + 1$$

Therefore,

$$C(n) = \frac{n(n-1)}{2}$$

Expanding gives a quadratic expression. Hence, best, average, and worst-case time complexities are all $O(n^2)$.

### Bubble Sort

In the worst case, the number of comparisons is approximately $n(n-1)/2$, giving $O(n^2)$. With the early-termination condition, its best case is $O(n)$, because only one pass is needed to establish that the array is sorted.

### Insertion Sort

In the best case, every element is already in position, so the algorithm makes $n-1$element comparisons and performs no shifts. Therefore, its best-case complexity is $O(n)$.

In the worst case, every element may need to move across the previously processed elements. The total work is proportional to $n(n-1)/2$, giving $O(n^2)$.

All three implementations use only a fixed number of additional variables. Their auxiliary space complexity is $O(1)$.

## 8. Design Decision

Insertion Sort is recommended for the fulfilment centre's nearly sorted data. It performs little work when new orders are added in small batches, and it sorts in place.

The recommendation is conditional: if the data is completely sorted and an optimized Bubble Sort is already implemented, Bubble Sort may also perform very little work. Actual selection should be validated using representative order data.

## Q2 – When Is Sequential Search an Appropriate Engineering Choice?

### 1. Experimental Design

Sequential Search checks records one by one until the required Tool ID is found or the list ends. The following experiments examine successful searches at different positions, unsuccessful searches, and different list sizes.

| Experiment | Number of records | Search condition | Expected comparisons |
|---|---|---|---|
| A | 5 | First record found | 1 |
| B | 5 | Middle record found | 3 |
| C | 5 | Last record found | 5 |
| D | 5 | Tool not present | 5 |
| E | 10 | Last record found | 10 |
| F | 20 | Tool not present | 20 |

### 2. Algorithm and Pseudocode

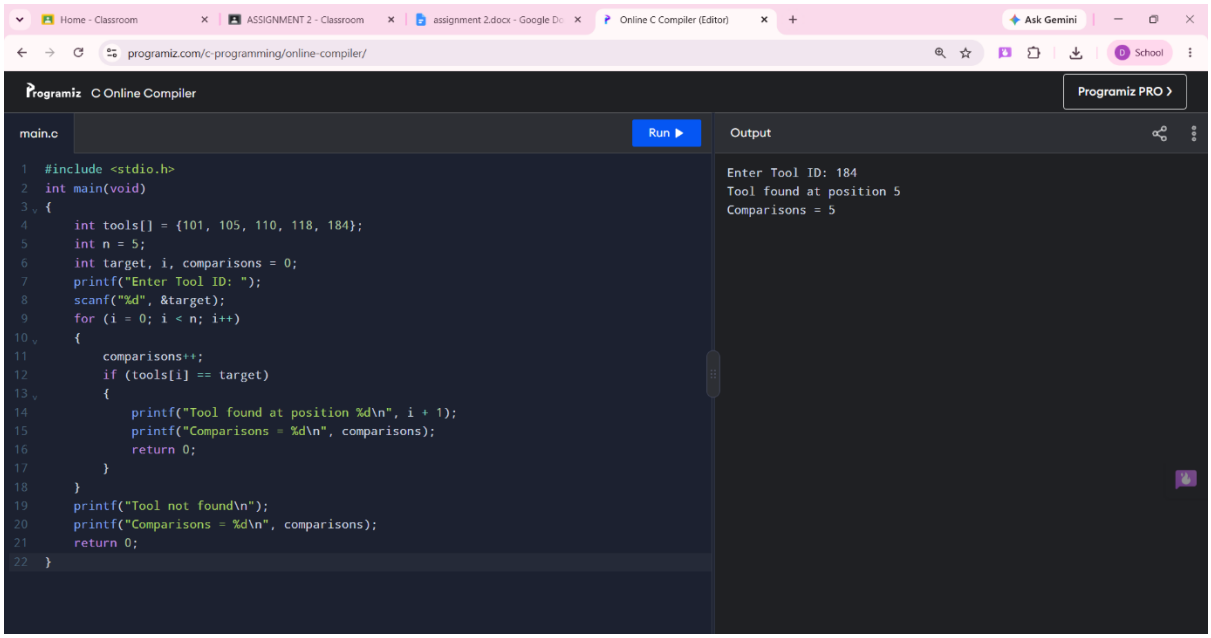
```
ALGORITHM SequentialSearch(A, n, target)
BEGIN
   FOR i ← 0 TO n - 1
      IF A[i] = target
         RETURN i
      END IF
   END FOR
   RETURN -1
END
```

**Line-by-line explanation**

- FOR i ← 0 TO n - 1: Visits every record in order.
- IF A[i] = target: Checks whether the current Tool ID matches the requested ID.
- RETURN i: Returns the position immediately when the tool is found.

- RETURN -1: Indicates that the tool does not exist in the list.

### 3. Complete C Program




**Code explanation**

- tools[] stores the available Tool IDs.
- n stores the number of records.
- target stores the Tool ID entered by the user.
- comparisons++ counts each Tool ID checked.
- if (tools[i] == target) tests for a match.
- return 0 terminates the program after a successful search.
- If the loop finishes, the requested Tool ID was not found.

### 4. Expected Output and Dry Run

Input:

Enter Tool ID: 184

Output:

Tool found at position 5

Comparisons = 5

Dry run:

| Position | Tool ID | Comparison |
|---|---|---|
| 1 | 101 | Not equal |
| 2 | 105 | Not equal |
| 3 | 110 | Not equal |
| 4 | 118 | Not equal |
| 5 | 184 | Match found |

The algorithm performs five comparisons because the target is the last record.

**5. Time and Space Complexity**

In the best case, the first record matches, requiring one comparison. Thus, the best-case time complexity is $O(1)$.

For a successful search where the target is equally likely to occupy any position, the average number of comparisons is:

$$C_{\text{avg}} = \frac{1 + 2 + \cdots + n}{n}$$

$$C_{\text{avg}} = \frac{n(n+1)}{2n} = \frac{n+1}{2}$$

This grows proportionally to $n$, giving average-case complexity $O(n)$. For an unsuccessful search, all $n$records must be examined. Therefore, the worst-case complexity is $O(n)$. The program uses a fixed number of additional variables, so auxiliary space complexity is $O(1)$.

**6. Engineering Decision**

Sequential Search is reasonable when the active list is small, records change frequently, and searches are relatively infrequent. It requires no sorting or indexing and works even when records are unordered. However, as the organisation expands to 20 facilities, both the number of records and the frequency of lookups may increase. Repeatedly checking every record can become expensive. At that scale, an indexed structure or hash table may be more suitable. The decision depends on the number of records, request frequency, memory constraints, and the required response time.

## Q3 – Engineer the Behaviour of Brute-Force String Matching

### 1. Experimental Design and Predictions

Brute-Force String Matching attempts to match a pattern at every possible starting position in a text. Three test conditions are designed.

| Test | Text | Pattern | Predicted behaviour |
|---|---|---|---|
| A | ABCDEF | AB | Few comparisons because the first position matches |
| B | AAAAAAAAAA | AAAAB | Many repeated comparisons before each mismatch |
| C | ABCABCABC | ABC | Several occurrences with successful character matches |

For Test B, the text contains repeated A characters. At each alignment, the first four pattern characters match, but the final B does not. This forces repeated comparisons.

### 2. Algorithm and Pseudocode

```
ALGORITHM BruteForceMatch(T, P)
BEGIN
  n ← length(T)
  m ← length(P)
  FOR i ← 0 TO n - m
    j ← 0
    WHILE j < m AND T[i + j] = P[j]
      j ← j + 1
    END WHILE
    IF j = m
      PRINT i
    END IF
```

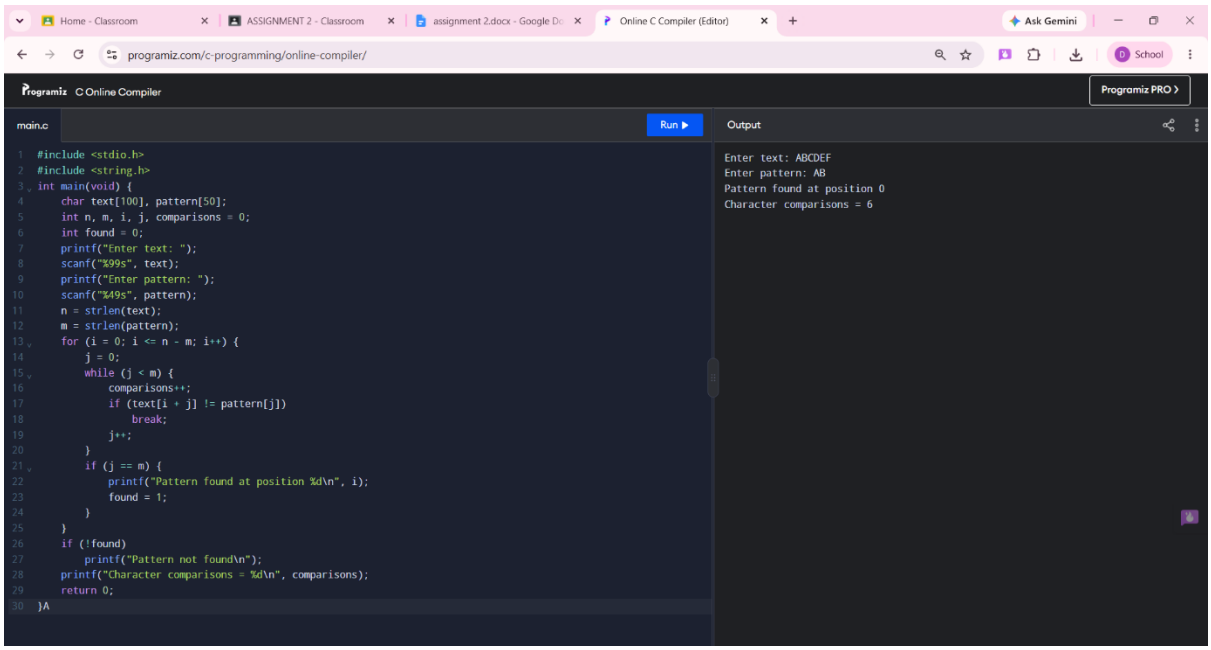
END FOR

END

**Pseudocode explanation**

- n ← length(T) obtains the text length.
- m ← length(P) obtains the pattern length.
- FOR i ← 0 TO n - m checks every valid starting position.
- j ← 0 begins comparison at the first pattern character.
- WHILE continues while characters match.
- j ← j + 1 advances to the next character after a match.
- IF j = m confirms that every pattern character matched.
- PRINT i reports the zero-based starting position.

### 3. Complete C Program




**Code explanation**

- strlen() determines the lengths of the text and pattern.
- The outer for loop examines each possible alignment.
- The inner while loop compares the pattern characters one by one.

- comparisons++ counts every character comparison, including a mismatch.
- break stops comparing at the first mismatch.
- j == m means the complete pattern matched.
- found records whether any occurrence exists.

**4. Expected Results**

**Test A**

Text: ABCDEF; Pattern: AB

The pattern matches at position 0, requiring two comparisons. The next alignment mismatches immediately. Continuing through all valid alignments gives the following result:

| Alignment | Comparisons |
|---|---|
| 0 | 2 |
| 1 | 1 |
| 2 | 1 |
| 3 | 1 |
| 4 | 1 |

**Test B**

Text: AAAAAAAAAA; Pattern: AAAAB

There are $10 - 5 + 1 = 6$alignments. At each alignment, four A characters match before the final B mismatches.

$$C = 6 \times 5 = 30$$

**Test C**

Text: ABCABCABC; Pattern: ABC

The pattern matches at positions 0, 3, and 6. The comparison count is:

$$3 + 1 + 1 + 3 + 1 + 1 + 3 = 13$$

The terms represent the comparisons at the seven valid alignments.

**5. Time and Space Complexity**

Let $n$be the text length and $m$the pattern length.

There are $n - m + 1$valid alignments. At each alignment, at most $m$character comparisons occur.

Therefore,

$$C(n, m) \leq (n - m + 1)m$$

The worst-case time complexity is $O(nm)$. When $m$grows proportionally to $n$, the bound becomes $O(n^2)$.

The best case is $O(n - m + 1)$, or $O(n)$when the pattern length is treated as fixed, because each alignment may fail on its first comparison.

The program stores the text and pattern in fixed-size arrays, so its auxiliary working space is $O(1)$for these fixed buffers. If the strings are dynamically sized, storing them requires $O(n + m)$total memory.

**6. Conclusion**

Input size alone does not fully explain the observed number of comparisons. Repeated prefixes can cause the same characters to be checked at many alignments. Therefore, both text length and pattern structure must be considered when evaluating brute-force matching.

## Q4 – Closest Pair and the Cost of Examining Every Possibility

**1. Algorithm Design**

Each drone reports its position as:

$$D_i = (x_i, y_i)$$

The objective is to find the two drones with the smallest Euclidean distance.

The brute-force approach examines every unique pair exactly once. To avoid duplicate comparisons, the inner loop begins at j = i + 1.

**2. Mathematical Derivation of Unique Pairs**

For $n$drones, the first drone can be paired with $n-1$drones, the second with $n-2$, and so on.

$$P = (n-1) + (n-2) + \cdots + 1$$

Using the arithmetic-series formula:

$$P = \frac{n(n-1)}{2}$$

Therefore, the number of unique pairs is:

$$\boxed{\frac{n(n-1)}{2}}$$

This avoids comparing drone A with drone B and then repeating the same comparison as drone B with drone A.

**3. Algorithm and Pseudocode**

```
ALGORITHM ClosestPair(P, n)
BEGIN
   minimum ← infinity
   FOR i ← 0 TO n - 2
      FOR j ← i + 1 TO n - 1
         distance ← EuclideanDistance(P[i], P[j])
         IF distance < minimum
            minimum ← distance
            first ← i
            second ← j
         END IF
      END FOR
   END FOR
```

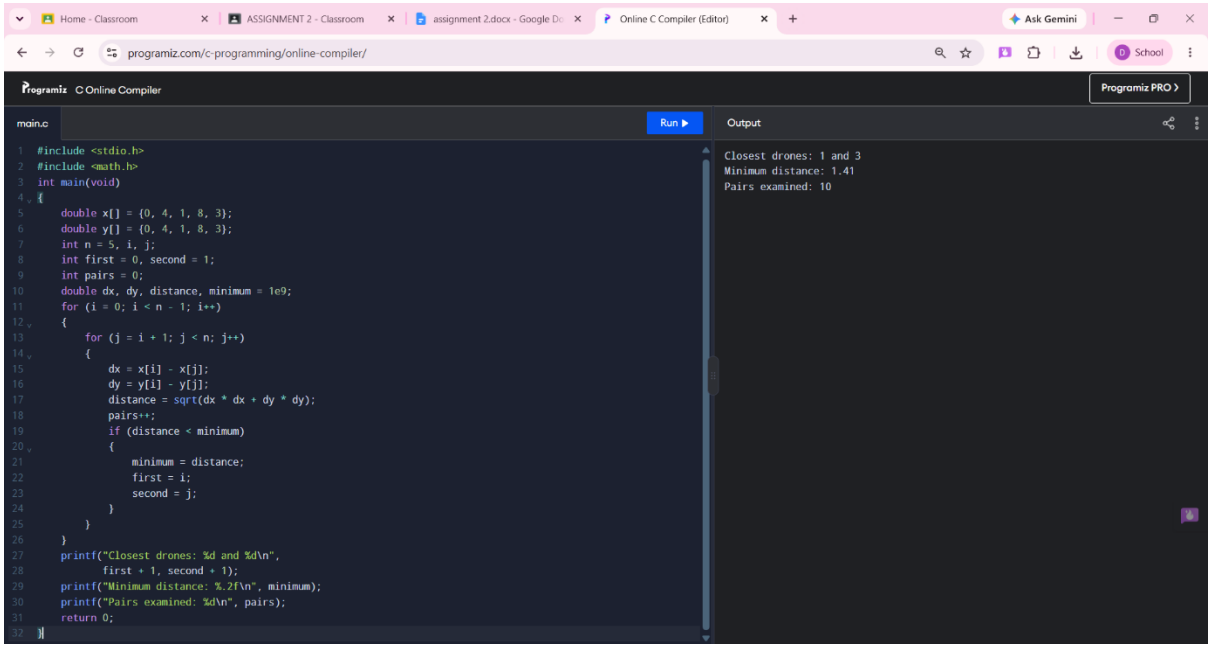
PRINT first, second, minimum

END

**Pseudocode explanation**

- minimum ← infinity ensures the first computed distance becomes the initial minimum.
- The outer loop chooses the first drone.
- The inner loop considers only drones after the first drone.
- EuclideanDistance calculates the distance between the two positions.
- The IF condition updates the closest pair when a shorter distance is found.
- The final statement reports the pair and its distance.

## 4. Complete C Program




**Code explanation**

- x[] and y[] store the coordinates. dx and dy store the differences between two points.
- sqrt(dx * dx + dy * dy) calculates Euclidean distance.
- pairs++ counts the unique pairs examined.
- minimum stores the shortest distance found so far.
- first and second store the indices of the closest drones.

**5. Expected Output**

For the given points, the closest pair is Drone 1 at $(0, 0)$and Drone 3 at $(1, 1)$.

Closest drones: 1 and 3

Minimum distance: 1.41

Pairs examined: 10

The pair count is:

$$P = \frac{5(5-1)}{2} = 10$$

**6. Scalability Investigation**

For three meaningful input sizes:

| **Number of drones $n$** | **Unique pairs** |
|---|---|
| 10 | 45 |
| 100 | 4,950 |
| 1,000 | 499,500 |

The number of pairs grows quadratically. Increasing the input from 100 to 1,000 drones multiplies the pair count by approximately 100. For a practical experiment, an acceptable response time might be defined as 100 milliseconds for one monitoring update. This is an engineering target, not a measured result. The actual largest acceptable input size must be established by running the program on the target computer.

**7. Time and Space Complexity**

The outer loop runs $n - 1$times. For each iteration, the inner loop examines the remaining points.

Thus,

$$T(n) \propto \frac{n(n-1)}{2}$$

The dominant term is $n^2$, giving $O(n^2)$time complexity.

Only a fixed number of coordinate differences, indices, and distance variables are stored, so auxiliary space complexity is $O(1)$, excluding the input coordinate arrays.

**8. Conclusion**

Brute-force Closest Pair is simple and guarantees the correct minimum for the examined points. However, its inefficiency originates from evaluating every possible pair. A redesigned strategy should reduce the number of candidate pairs or exploit spatial relationships to avoid unnecessary distance calculations.

## Q5 – Derive a Convex Hull Algorithm from a Geometric Principle

**1. Geometric Decision Rule**

The convex hull is the smallest convex boundary containing all given points. It resembles a tight rubber band stretched around the outermost points. For two candidate points $A(x_1, y_1)$and $B(x_2, y_2)$, calculate the orientation of another point $C(x_3, y_3)$:

$$S = (x_2 - x_1)(y_3 - y_1) - (y_2 - y_1)(x_3 - x_1)$$

Interpretation:

- $S > 0$: $C$lies on one side of the directed line.
- $S < 0$: $C$lies on the opposite side.
- $S = 0$: The points are collinear.

If every other point lies on the same side of a candidate line, that line supports the point set and may define a hull edge.

**2. Algorithm Development**

1. Select every possible pair of points.
2. Calculate the orientation of every remaining point relative to that pair.
3. Record whether positive and negative orientations occur.
4. Accept the pair as a hull edge if all non-collinear points lie on one side.
5. Connect the accepted edges to construct the boundary.

6. Handle collinear points consistently, retaining only extreme endpoints when appropriate.

**3. Pseudocode**

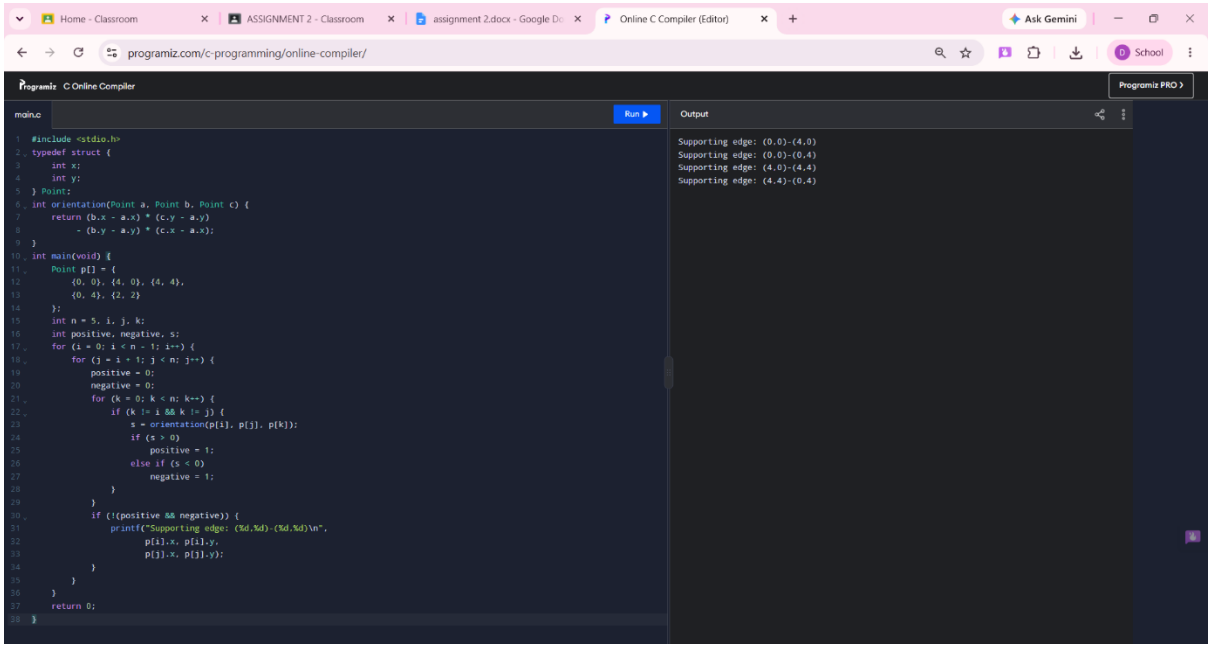
```
ALGORITHM BruteForceHull(P, n)
BEGIN
  FOR i ← 0 TO n - 2
    FOR j ← i + 1 TO n - 1
      positive ← FALSE
      negative ← FALSE
      FOR k ← 0 TO n - 1
        IF k != i AND k != j
          s ← Orientation(P[i], P[j], P[k])
          IF s > 0
            positive ← TRUE
          ELSE IF s < 0
            negative ← TRUE
          END IF
        END IF
      END FOR
      IF NOT (positive AND negative)
        PRINT candidate edge (i, j)
      END IF
    END FOR
  END FOR
END
```

**Pseudocode explanation**

- The first two loops select every candidate edge.
- positive and negative record which sides contain points.
- The third loop examines all other points.
- Orientation() calculates the side of the candidate line.
- If both flags become true, points exist on both sides, so the candidate is not a supporting edge.
- Otherwise, the candidate is accepted as a supporting edge.

**Important limitation:** This procedure identifies supporting segments, but a complete implementation must handle collinear points and order the accepted edges into a boundary without duplicates. The following C program implements the decision test and prints candidate edges; it is not, by itself, a complete ordered polygon visualizer.

**4. C Program**




**Code explanation**

- Point stores the x- and y-coordinates.
- orientation() calculates the signed area expression that determines the side of a line.
- The outer loops choose candidate point pairs.

- The innermost loop checks all remaining points.
- positive and negative record the presence of points on each side.
- A candidate is printed if points do not occur on both sides.

**5. Test Configurations**

**Test A: Boundary and interior points**

$$(0,0),(4,0),(4,4),(0,4),(2,2)$$

The first four points form a square. The point $(2,2)$is interior and should not become a hull vertex.

**Test B: Collinear boundary points**

$$(0,0),(2,0),(4,0),(4,4),(0,4)$$

Three points lie along the bottom boundary. The correct convex hull uses the extreme endpoints $(0,0)$and $(4,0)$, rather than treating $(2,0)$as a necessary vertex.

**6. Correctness Argument**

If all points lie on one side of a candidate line, the line supports the point set. A supporting line can contain an edge of the convex hull because the point set never crosses to the opposite side. If points lie on both sides, the candidate segment cannot be a hull edge because the point set crosses its supporting line. Collinear points require special handling. Multiple point pairs may lie on the same supporting line, so a complete hull construction must select the extreme endpoints and arrange edges in boundary order.

**7. Time and Space Complexity**

There are

$$\frac{n(n-1)}{2}$$

candidate pairs. For each pair, the algorithm checks up to $n-2$other points.

Therefore,

$$T(n) \propto \frac{n(n-1)}{2}(n-2)$$

The dominant term is cubic, giving $O(n^3)$time complexity. The implementation uses a fixed number of variables for each candidate pair, so its auxiliary working space is $O(1)$, excluding the input points.

**8. Conclusion**

The geometric orientation test provides a computational method for identifying supporting lines. However, complete convex hull construction also requires careful handling of collinear points, duplicate edges, and boundary ordering.

## Q6 – Exhaustive Search and Combinatorial Growth

**1. Problem Representation**

Each package has a weight and a humanitarian value. The objective is to select a subset of packages that maximizes total value without exceeding the helicopter's payload limit.

Let:

- $w_i$= weight of package $i$
- $v_i$= value of package $i$
- $W$= maximum payload

A subset is feasible when:

$$\sum w_i \leq W$$

Among all feasible subsets, the algorithm chooses the subset with the greatest total value.

**2. Algorithm and Pseudocode**

Every package has two possibilities: selected or not selected. Therefore, $n$packages generate $2^n$subsets.

ALGORITHM ExhaustiveKnapsack(weight, value, n, W)

BEGIN

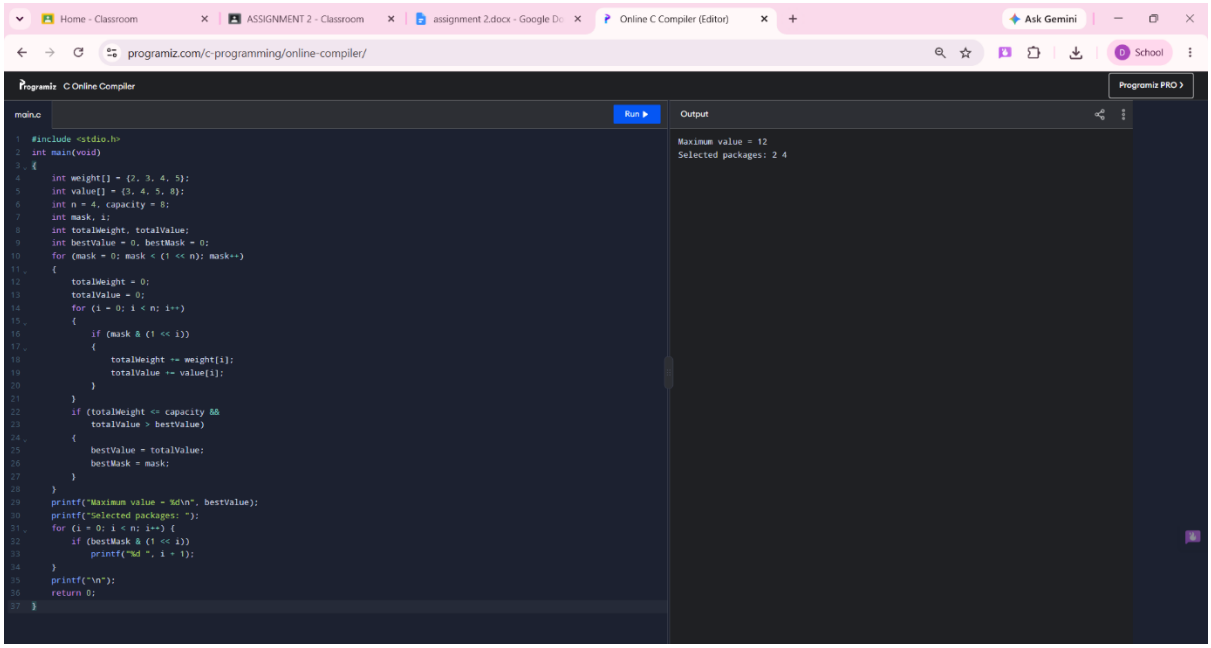
bestValue ← 0

```
    bestMask ← 0
    FOR mask ← 0 TO 2^n - 1
        totalWeight ← 0
        totalValue ← 0
        FOR i ← 0 TO n - 1
            IF bit i of mask is 1
                totalWeight ← totalWeight + weight[i]
                totalValue ← totalValue + value[i]
            END IF
        END FOR
        IF totalWeight <= W AND totalValue > bestValue
            bestValue ← totalValue
            bestMask ← mask
        END IF
    END FOR
    PRINT bestMask, bestValue
END
```

**Pseudocode explanation**

- bestValue stores the highest feasible value found.
- bestMask records which packages form the current best solution.
- The outer loop enumerates every subset.
- The binary representation of mask identifies selected packages.
- The inner loop calculates total weight and value.
- The feasibility condition rejects subsets exceeding the payload.

## 3. Complete C Program




### Code explanation

- 1 << n calculates $2^n$, the number of subsets.
- mask represents a particular subset.
- mask & (1 << i) checks whether package $i$ is included.
- totalWeight and totalValue accumulate the selected packages' properties.
- The if condition verifies feasibility and improves the best value.
- The final loop prints the selected package numbers.

## 4. Correctness Verification

The packages are:

| Package | Weight | Value |
|---|---|---|
| 1 | 2 | 3 |
| 2 | 3 | 4 |
| 3 | 4 | 5 |
| 4 | 5 | 8 |

Capacity:

$$W = 8$$

Selecting packages 2 and 4 gives:

$$\text{Weight} = 3 + 5 = 8$$
$$\text{Value} = 4 + 8 = 12$$

Selecting packages 1, 2, and 3 gives weight $2 + 3 + 4 = 9$, which exceeds capacity. Packages 1 and 4 give value 11, while packages 2 and 3 give value 9.

Thus, the optimum is packages 2 and 4, with total value 12.

**5. Expected Output**

Maximum value = 12

Selected packages: 2 4

**6. Growth Investigation**

The number of subsets is $2^n$.

| **Packages** $n$ | **Candidate subsets** $2^n$ |
|---|---|
| 5 | 32 |
| 10 | 1,024 |
| 15 | 32,768 |

Doubling the number of packages does not merely double the work. Every additional package doubles the number of subsets. For example:

$$2^{10} = 1{,}024$$
$$2^{15} = 32{,}768$$

Adding five packages multiplies the number of candidates by 32.

**7. Time and Space Complexity**

The algorithm examines $2^n$subsets. For each subset, it may inspect all $n$packages to calculate total weight and value.

Therefore,

$$T(n) = O(n2^n)$$

The $2^n$factor arises from subset enumeration, while the $n$factor arises from examining packages within each subset. The algorithm stores a fixed number of totals and indices, giving $O(1)$auxiliary space beyond the input arrays.

**8. Deployment Decision**

For a simple experiment, a response-time target of 100 milliseconds may be selected, subject to the actual emergency-planning requirements. The table predicts candidate counts, but it does not establish a measured time limit. The program must be tested on the deployment computer to identify the largest package count that consistently meets the target. Exhaustive Search is appropriate for small instances where verifying the exact optimum is important. As the package count increases, a different strategy may be required to control computational effort.

## Q7 – Diagnose, Redesign and Validate

**Selected solution:** Exhaustive Search for humanitarian package selection.

This solution is selected because its candidate space grows exponentially, creating a clear computational bottleneck that can be investigated and redesigned.

**A. Diagnose the Computational Bottleneck**

The original algorithm generates all $2^n$package combinations. For every combination, it examines up to $n$packages to calculate total weight and value.

Therefore, the number of package-inspection operations is proportional to:

$$n2^n$$

For 10 packages, there are 1,024 subsets. For 15 packages, there are 32,768 subsets.

The bottleneck is the repeated examination of packages across every possible combination. As the package count increases, the number of combinations doubles for every additional package.

**B. Define the Design Objective**

The redesign aims to reduce repeated package calculations while preserving the maximum feasible humanitarian value.

The measurable objectives are:

1. Return the same optimal value as Exhaustive Search.
2. Avoid recalculating identical subproblems.
3. Record the number of states processed.
4. Compare the original and redesigned methods using the same input data.

**C. Design the Revised Computational Strategy**

A suitable redesign is Dynamic Programming using a capacity-based table.

Instead of independently evaluating every subset, the algorithm stores the best value achievable for each capacity after considering a certain number of packages.

Let $DP[i][c]$denote the maximum value achievable using the first $i$packages with capacity $c$.

If package $i$is too heavy, it cannot be selected:

$$DP[i][c] = DP[i-1][c]$$

Otherwise, the algorithm compares two possibilities:

$$DP[i][c] = \max\begin{cases} DP[i-1][c] \\ v_i + DP[i-1][c-w_i] \end{cases}$$

The first option excludes the package. The second includes it and adds its value to the best feasible result for the remaining capacity.

**Pseudocode**

```
ALGORITHM DynamicKnapsack(weight, value, n, W)
BEGIN
  INITIALIZE DP[0..n][0..W] TO 0
  FOR i ← 1 TO n
    FOR c ← 0 TO W
      DP[i][c] ← DP[i-1][c]
      IF weight[i-1] <= c
```

```
            candidate ← value[i-1]
                        + DP[i-1][c-weight[i-1]]
            IF candidate > DP[i][c]
               DP[i][c] ← candidate
            END IF
         END IF
      END FOR
   END FOR
   RETURN DP[n][W]
END
```

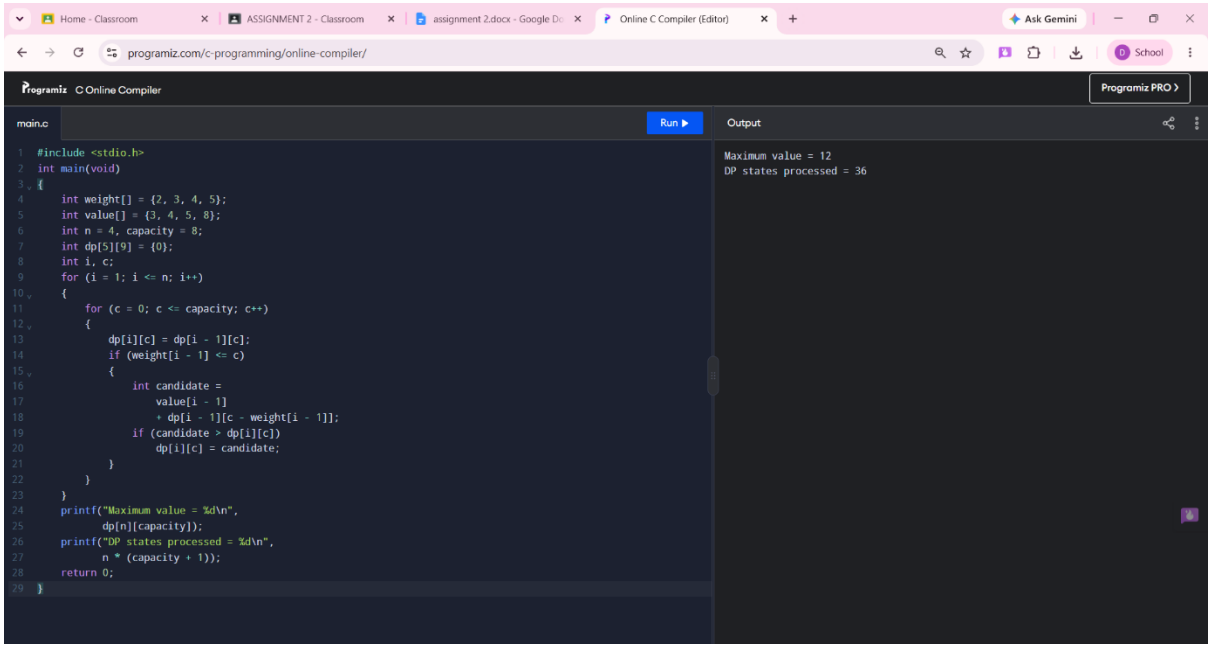
**Pseudocode explanation**

- The table begins with zero values because no packages produce zero humanitarian value.
- The outer loop introduces packages one at a time.
- The inner loop examines every capacity from zero to the maximum payload.
- DP[i][c] ← DP[i-1][c] represents excluding the current package.
- The weight condition checks whether the package fits.
- candidate calculates the value obtained by including the package.
- The IF condition retains whichever choice gives greater value.
- DP[n][W] contains the optimum after all packages are considered.

**D. Predict Before Implementing**

The prediction is that Dynamic Programming will produce the same optimal value as Exhaustive Search while reducing the number of states examined for instances with a moderate payload limit. The expected complexity changes from $O(n2^n)$to $O(nW)$, where $W$is the capacity limit. This improvement is especially useful when the payload limit is moderate, although performance still depends on the numerical size of $W$.

## E. Implement and Validate the Redesign




### Code explanation

- dp[5][9] stores results for five row states and nine capacity states.
- The outer loop processes the four packages.
- The inner loop evaluates each capacity from zero to eight.
- The first assignment copies the result obtained without the current package.
- The candidate calculation evaluates the result if the package is included.
- The comparison keeps the greater value.
- The final cell contains the optimum for all four packages and capacity eight.

### Expected output

Maximum value = 12

DP states processed = 36

The original algorithm examines $2^4 = 16$subsets. The redesigned program fills $4(8 + 1) = 36$package-capacity states. This small example does not show fewer raw iterations because the instance is tiny and the measurements count different kinds of work. The benefit of Dynamic Programming is avoiding repeated subset evaluation as the solution space grows.

**F. Defend the Final Design**

Dynamic Programming is justified when the payload capacity is moderate and many combinations produce overlapping subproblems.

Its time complexity is:

$$T(n, W) = O(nW)$$

The table contains $(n + 1)(W + 1)$cells. Therefore, its space complexity is:

$$S(n, W) = O(nW)$$

The redesigned method uses more memory than the constant-space exhaustive implementation, but it avoids explicitly enumerating all subsets.

The original Exhaustive Search may remain preferable for very small package sets because it is simple and straightforward to verify. Dynamic Programming is more appropriate when the original method's repeated computation becomes costly.

**Final conclusion:** The redesign addresses the diagnosed bottleneck by reusing previously computed results. Both methods should be tested on identical instances, and the final deployment boundary should be determined from actual execution measurements and the required response time.